

Project Report: Enhancing LLM Inference with GraphRAG

Daniel Granström
Aalto University
Espoo, Finland
daniel.granstrom@aalto.fi

Joonatan Korpela
Aalto University
Espoo, Finland
joonatan.korpela@aalto.fi

Mikael Siidorow
Aalto University
Espoo, Finland
mikael.siidorow@aalto.fi

Abstract

The course project is available at <https://github.com/dcgran/CS-E4780-project> [1].

- GraphRAG implementation on Nobel Laureates dataset
- Text2Cypher accuracy improvements + performance optimization
- Key metrics: accuracy, latency, cache hit rate, memory usage
- GitHub repository: <https://github.com/dcgran/CS-E4780-project> [1]

Keywords

Large Language Models, Retrieval-Augmented Generation

ACM Reference Format:

Daniel Granström, Joonatan Korpela, and Mikael Siidorow. 2025. Project Report: Enhancing LLM Inference with GraphRAG. In *Proceedings of CS-E4780 Course Project Evaluation Tables (CS-E4780)*. ACM, New York, NY, USA, 3 pages.

1 Introduction

Retrieval-Augmented Generation (RAG) systems enhance Large Language Model (LLM) inference by incorporating external knowledge sources to improve factual accuracy and reduce hallucinations. Traditional RAG approaches rely on semantic similarity search over unstructured text chunks, which works well for many tasks but struggles to capture the rich relational structure underlying complex domains such as scientific collaboration networks, enterprise knowledge bases, and multi-hop reasoning problems. GraphRAG addresses this limitation by integrating graph representations of knowledge into the retrieval process [2], enabling LLMs to reason over structured relationships and perform multi-hop queries that would be difficult to express through text-based retrieval alone.

This project implements a GraphRAG system using the Nobel Laureates dataset, which contains mentorship relationships spanning from the 16th century to 2021, enriched with biographical data from the official Nobel Prize API. The dataset is stored in Kuzu, a graph database that supports efficient pattern matching through the Cypher query language. The system pipeline consists of four main stages: schema pruning to identify relevant graph structure, Text2Cypher generation to translate natural language questions into Cypher queries, query execution against the graph database, and response generation by the LLM based on retrieved results.

We address two primary objectives in this work. First, we enhance the Text2Cypher component to improve query generation accuracy through three techniques: (1) dynamic few-shot exemplar selection based on semantic similarity to the input question, (2) a self-refinement loop that validates generated queries using

dry-run execution and repairs syntax errors iteratively, and (3) rule-based post-processing to enforce common patterns such as case-insensitive string matching. Second, we optimize system performance by implementing an LRU cache keyed on question and schema hashes, and conduct comprehensive benchmarking to characterize latency across all pipeline stages and quantify memory utilization.

Our evaluation includes an ablation study examining all meaningful combinations of Text2Cypher enhancements, quantifying the accuracy-latency trade-offs inherent in each approach. We measure both syntax correctness and semantic correctness against a diverse test set, and analyze cache performance under different workload patterns. The results provide practical insights into which optimizations yield the best balance between response quality and system efficiency.

The remainder of this report is organized as follows. Section 2 describes the system architecture and design decisions. Section 3 details the implementation of both tasks, including testbed setup and measurement methodology. Section 4 presents our experimental evaluation with comprehensive performance analysis. Section 5 discusses key findings and lessons learned, and Section 6 concludes.

2 System Architecture

2.1 GraphRAG Pipeline

Our GraphRAG system implements a five-stage pipeline that transforms natural language questions into structured graph queries and generates contextually grounded responses. Figure ?? illustrates the complete workflow.

Stage 1: Schema Retrieval. The system begins by extracting the complete graph schema from Kuzu, including all node types, relationship types, and their associated properties. This schema provides a comprehensive structural description of the knowledge graph that will be used in subsequent stages.

Stage 2: Schema Pruning. Given a user question, the system uses an LLM to identify which subset of the schema is relevant to answering that question. This pruning step reduces the context size for downstream stages, improving both accuracy and latency. The pruned schema is serialized and hashed to serve as part of the cache key, enabling reuse across questions that involve the same graph structure.

Stage 3: Text2Cypher Generation. The pruned schema and user question are passed to the Text2Cypher component, which generates a syntactically and semantically valid Cypher query. This stage incorporates three enhancement mechanisms (detailed in Section 3.2): similarity-based few-shot exemplar selection, self-refinement with validation feedback, and rule-based post-processing. The generated query is cached using a composite key of question hash and schema hash.

Stage 4: Query Execution. The generated Cypher query is executed against the Kuzu database. Kuzu’s columnar storage and optimized query engine enable efficient pattern matching over the graph structure. The query results are returned as structured data (e.g., lists of nodes, relationships, or aggregated values).

Stage 5: Response Generation. Finally, an LLM synthesizes a natural language response based on the original question, the generated query, and the structured results from the database. This stage translates the retrieved facts into a coherent, human-readable answer that directly addresses the user’s information need.

2.2 Knowledge Graph Schema

The Nobel Laureates knowledge graph represents prize winners, their institutional affiliations, and geographic metadata. The graph is constructed from the official Nobel Prize API, which provides comprehensive biographical and institutional data for all laureates from 1901 to 2021 across four prize categories. Figure 1 illustrates the complete schema.

Node Types. The schema defines six entity types:

- **Scholar:** Represents Nobel Prize laureates. Properties include id, knownName, fullName, gender, birthDate, deathDate, and scholar_type (set to “laureate” for all nodes from the API data).
- **Prize:** Represents individual Nobel Prize awards in a given year and category. Properties include prize_id (composite key of year and category), awardYear, category (physics, chemistry, medicine, economics), dateAwarded, motivation, prizeAmount, and prizeAmountAdjusted.
- **Institution:** Academic and research institutions affiliated with scholars. Properties include name (standardized current name).
- **City:** Geographic locations representing cities associated with scholar birthplaces and institutional locations. Properties include name and state.
- **Country:** National-level geographic entities. Properties include name.
- **Continent:** Continental-level geographic entities for regional analysis. Properties include name.

Relationship Types. The schema defines seven relationship types connecting entities:

- **WON:** Connects a Scholar to a Prize they received. Includes a portion property indicating the share of the prize (e.g., “1/2” for shared prizes).
- **BORN_IN:** Connects a Scholar to their birth City.
- **DIED_IN:** Connects a Scholar to the City where they died (defined but sparsely populated in current data).
- **AFFILIATED_WITH:** Connects a Scholar to an Institution they were associated with at the time of receiving the prize.
- **IS_LOCATED_IN:** Connects an Institution to its host City.
- **IS_CITY_IN:** Connects a City to its Country.
- **IS_COUNTRY_IN:** Connects a Country to its Continent.

This schema enables multi-hop queries such as: “How many Physics laureates were born in cities located in European countries?” or “Which institutions affiliated with the most Chemistry prize winners are located in the United States?” The hierarchical geographic

structure (City → Country → Continent) supports analysis at multiple spatial granularities, while the institutional affiliations enable studies of knowledge production across research organizations.

2.3 Component Interactions

- Component interactions
- Design decisions and rationale for each task

3 Implementation

3.1 Testbed Setup

- Hardware specifications
- Software environment (Python, Kuzu, LLM API)
- Test query dataset

3.2 Task 1: Text2Cypher Enhancement

- Dynamic few-shot exemplar selection
- Self-refinement loop (generate → validate → repair)
- Rule-based post-processing
- Implementation details for each component

3.3 Task 2: Caching and Performance

- LRU cache design (key computation, data structure)
- Benchmarking infrastructure
- Memory measurement methodology (process-level, object-level, database-level)

4 Performance Evaluation

4.1 Task 1: Text2Cypher Accuracy and Latency

Table 1 presents the ablation study showing accuracy-latency trade-offs.

- Analysis of accuracy-latency trade-offs
- Which combinations are most effective?
- Example success/failure cases

4.2 Task 2: Performance and Efficiency

Table 2 shows the pipeline stage latency breakdown.

- Cache hit rate analysis
- Speedup quantification
- Performance visualization (flamegraph/bar chart)
- Bottleneck identification

5 Discussion

- Key findings from ablation study
- Optimal configuration recommendation
- Challenges encountered and solutions
- Lessons learned

6 Conclusion

- Summary of Task 1 and Task 2 achievements
- Key contributions
- Practical insights

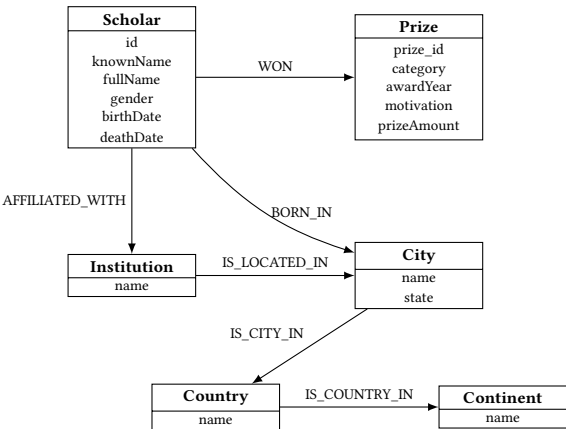


Figure 1: Knowledge graph schema showing node types and relationships in the Nobel Laureates dataset. The schema supports multi-hop queries across prize winners, their institutional affiliations, and hierarchical geographic locations.

Table 1: Ablation Study - Accuracy vs Latency Trade-offs

Configuration	Syntax Correct (%)	Semantic Correct (%)	Avg Latency (ms)	Avg Iterations
Baseline (static few-shot)	XX	XX	XXX	1.0
Only Few-shot selection	XX	XX	XXX	1.0
Only Self-refinement	XX	XX	XXX	X.X
Only Post-processing	XX	XX	XXX	1.0
Few-shot + Post-processing	XX	XX	XXX	1.0
Few-shot + Refinement	XX	XX	XXX	X.X
Refinement + Post-processing	XX	XX	XXX	X.X
All enhancements	XX	XX	XXX	X.X

Table 2: Pipeline Stage Latency Breakdown (ms)

Stage	Without Cache	With Cache (hit)	With Cache (miss)
Schema Pruning	XX	XX	XX
Text2Cypher	XXX	<1	XXX
Query Execution	XX	XX	XX
Response Generation	XX	XX	XX
Total	XXX	XX	XXX

References

[1] Daniel Granström, Joonatan Korpela, and Mikael Siidorow. 2025. Enhancing LLM Inference with GraphRAG. <https://github.com/dcgran/CS-E4780-project>. Accessed: 2025-10-10.

[2] Haoyu Han, Yu Wang, Harry Shomer, Kai Guo, Jiayuan Ding, Yongjia Lei, Mahantesh Halappanavar, Ryan A. Rossi, Subhabrata Mukherjee, Xianfeng Tang, Qi He, Zhigang Hua, Bo Long, Tong Zhao, Neil Shah, Amin Javari, Yinglong Xia, and Jiliang Tang. 2025. Retrieval-Augmented Generation with Graphs (GraphRAG). arXiv:2501.00309 [cs.IR] <https://arxiv.org/abs/2501.00309>